

Agentic RAG Chatbot

Fejlesztői dokumentáció

Gyakorlati útmutató a helyi frontend-dokumentációs asszisztens fejlesztéséhez,
futtatásához és bővítéséhez.

Python 3.12 / LangGraph / Streamlit / Ollama / Chroma

Működő prototípus · 0.1.0 verzió

A RAG-megbízhatósági frissítést és 2026. október 4-i méréseit tartalmazza; a 2026. október 3-i mérések az alapértéket adják.

2026. október 4.

Tartalomjegyzék

01	A projekt célja és felépítése	3
02	Tájékozódás a kódban	4
03	Helyi fejlesztői környezet	5
04	Fake mód és parancssori használat	6
05	Futtatás Docker Compose-zal	7
06	Futásidejű architektúra	8
07	Irányítás, tervezés és ellenőrzés	9
08	Állapotszerződések és Python API	10
09	Korpuszletöltés és előfeldolgozás	11
10	Az index életciklusa és migrációja	12
11	Hibrid RAG-keresés	13
12	Determinisztikus eszközök	14
13	Modellek és tárolás beállításai	15
14	A munkafolyamat beállításai	16
15	Felület és nyomkövetés	17
16	Tesztelés és folyamatos integráció	18
17	Az alkalmazás bővítése	19
18	Funkcionális értékelés	20
19	Teljesítmény és mért eredmények	21
20	Hibaelhárítás és üzemeltetés	22
21	Karbantartás és forrásjegyzék	23

A parancsokat a projekt gyökérkönyvtárából kell futtatni. A példák jelölik a használt parancsértelmezőt. A kódazonosítók, környezeti változók és fájlvonalak mindkét nyelvi változatban azonosak.

01 A projekt célja és felépítése

Az Agentic RAG Chatbot egy Pythonban megvalósított frontend-fejlesztői asszisztens prototípusa. Letöltött, rögzített verziójú hivatalos dokumentáció alapján válaszol, a pontosan kiszámítható vagy táblázatból kikereshető tényekhez pedig helyi, determinisztikus eszközöket használ.

A tudásbázis az MDN Web Docs, a React, a Vue, a Next.js, a Nuxt és a TypeScript Handbook anyagait tartalmazza. A felhasználó angolul és magyarul is kérdezhet. A keresés angolra írja át a kérdést; a szokásos válaszadás az irányító által azonosított nyelvet követi. A szó szerinti eszközkiemenetek és egyes tartaléküzenetek a megvalósítás szerinti angol szöveget őrzik meg.

Mit mutat be a rendszer?

- Hét fő LangGraph-csomópont, feltételes irányítás és párhuzamosan, önállóan végrehajtott részfeladatok.
- Külön, négy csomópontos RAG-részgráf, amelyet a tudásbázis-kereső eszköz hív meg.
- Helyi Ollama-következtetés, CPU-n futó beágyazások és tartós Chroma-index.
- Streamlit beszélgetőfelület a végrehajtási lépések és a felhasznált bizonyítékok megjelenítésével.
- Megismételhető adatbetöltés, offline tesztek, funkcionális értékelés és terheléses mérés.

A megvalósítás lényeges határai

Az alkalmazás prototípus: beszélgetőfelületet és parancssori eszközt biztosít, külön nyilvános REST API nélkül. A gráfállapot minden kérésnél új: nincs LangGraph-ellenőrzőpont vagy tartós beszélgetéstár. A felület a lezárt beszélgetési előzményeket adja át a következő hívásnak.

A fake szolgáltatók fejlesztési teszthelyettesítők. Modellszerver nélkül ellenőrzik a végrehajtási útvonalakat; válaszaik és keresési pontszámaik nem igazolják a modell minőségét. A korpusz első letöltéséhez fake szolgáltatókkal is hálózati hozzáférés kell.

A projekt kódja MIT-licencű. A forrásjegyzék külön rögzíti a letöltött dokumentációk licenceit. A korpuszt a rendszer letölti, nem tárolja a Git-adattárban.

Forrás: `README.md`, `pyproject.toml`, `agent/graph.py`, `agent/state.py`, `data/sources.toml`.

02 Tájékozódás a kódban

Az alkalmazáskód a `src/agentik_rag/` könyvtárban található. A telepített csomagot importáld; az alkalmazáskódban kerül a `sys.path` módosítását. A konzolos belépési pont: `agentik-rag = agentik_rag.cli:main`.

Hely	Feladat
<code>config.py, errors.py</code>	Ellenőrzött beállítások és közös alkalmazáskivételek.
<code>cli.py, __main__.py</code>	Parancsok, argumentumellenőrzés, naplózás és kilépési kódok.
<code>llm.py, embeddings.py</code>	Valós és fake szolgáltatók, helyi modelladapterek.
<code>agent/graph.py, routing.py</code>	A fő gráf kapcsolatai és tiszta feltételes irányítása.
<code>agent/nodes.py, prompts.py, state.py</code>	Csomópontműködés, strukturált promptok és állapotszerződések.
<code>agent/tools.py</code>	Keresőeszköz és a három determinisztikus eszköz burkolója.
<code>agent/contrast.py, specificity.py, compat.py</code>	Szakterületi számítások és kompatibilitási adatok lekérdezése.
<code>rag/</code>	Kérdésátírás, keresés, szűrés, hivatkozott kontextus és BM25-fúzió.
<code>ingestion/</code>	Forrásjegyzék, letöltés, tisztítás, darabolás, indexelés és induláskori előkészítés.
<code>ui/</code>	Streamlit-belépési pont és megjelenítő komponensek.
<code>evaluation/, loadtest/, reports.py, tracing.py</code>	Adatkészletek, metrikák, jelentések és közös mérési infrastruktúra.
<code>tests/, .github/workflows/ci.yml</code>	Offline ellenőrzés és folyamatos integráció.

Adatok és infrastruktúra

A `data/sources.toml` és az értékelési JSONL-fájlok verziókezelt bemenetek. A `data/raw/` letöltött forrásai és a `data/chroma_db/` vektorai generált, Git által figyelmen kívül hagyott adatok. A `data/eval/results/` korábbi mérési jelentéseket tárol.

A Python-környezetet a `pyproject.toml`, az `uv.lock` és a `.python-version` határozza meg. A konténerek működését a `Dockerfile`, a `compose.yaml` és az opcionális `compose.gpu.yaml` írja le. A `.streamlit/config.toml` a felület témáját állítja be.

A következő fejezetek útvonalai a `src/agentik_rag/` könyvtárhoz képest értendők, kivéve a `data/`, `docs/`, `tests/` kezdetű és a projekt gyökerében lévő fájlokat.

03 Helyi fejlesztői környezet

Python 3.12-t használj: a `pyproject.toml` követelménye `>=3.12,<3.13`. Telepítsd a Gitet és az `uv`-t; valós válaszokhoz Ollama is szükséges. A `lockfájl` CPU-s PyTorchot választ, hogy a GPU erőforrásai a beszélgetőmodell számára maradjanak elérhetők.

A projekt előkészítése

Az alábbi parancsok PowerShellben és POSIX parancsértelmezőben is használhatók. A klónozás után a projekt gyökérkönyvtárából futtasd őket.

```
git clone https://github.com/Csabikahhh/agentic-rag-chatbot-poc.git
cd agentic-rag-chatbot-poc
uv sync --locked
uv run agentic-rag --help
uv run agentic-rag config
```

Az `uv sync --locked` telepíti a projektet és a fejlesztői eszközöket, de eltérés esetén nem írja át észrevétlenül a `lockfájl`t. Windowson a konfiguráció másolásához használd a `Copy-Item .env.example .env` parancsot; Linuxon vagy macOS-en a `cp .env.example .env` parancsot.

A valós helyi rendszer indítása

Indítsd el az Ollama asztali alkalmazást, vagy külön terminálban futtasd az `ollama serve` parancsot. Ezután:

```
ollama pull qwen3.5:4b
uv run agentic-rag ingest --download
uv run agentic-rag serve
```

Nyisd meg a `http://localhost:8501` címet. A helyi Ollama alapértelmezett címe `http://localhost:11434`. Az első adatbetöltés letölti a rögzített dokumentációt és a beágyazási modellt; a folyamat első keresése betölti a beágyazási modellt.

A `serve` alapértelmezés szerint induláskori adatbetöltést végez. A fenti külön `ingest --download` lépés láthatóvá és könnyebben vizsgálhatóvá teszi az előkészítést. A közvetlen `uv run streamlit run src/agentic_rag/ui/app.py` indítás nem végzi el a CLI előkészítését; ehhez előbb készítsd el az indexet.

Előfeltételek és parancsok: `README.md`; Python-verzió és belépési pont: `pyproject.toml`; indulás: `cli.py`.

04 Fake mód és parancssori használat

A fake mód előre megírt beszélgetőszolgáltatót és determinisztikus, hash-alapú beágyazásokat használ. Mindkét szolgáltatót együtt állítsd át; helyi váltáskor külön indexkönyvtárat használj, hogy a fake index ne cserélje le a valódi modell indexét.

PowerShell-példa

```
$env:LLM_PROVIDER = "fake"
$env:EMBEDDING_PROVIDER = "fake"
$env:CHROMA_DIR = "data/chroma_db_fake"
uv run agentic-rag ingest --download
uv run agentic-rag serve
```

POSIX parancsértelmezőben előbb futtasd az `export LLM_PROVIDER=fake EMBEDDING_PROVIDER=fake` és az `export CHROMA_DIR=data/chroma_db_fake` parancsot. Ezután ugyanazok az indítóparancsok következnek. Ha a korpusz már megvan, a `sima ingest` nem kezdeményez letöltést.

A PowerShell környezeti változói a terminálmunkamenetben megmaradnak, és felülírják a `.env` értékeit. A valódi alapértékekhez való visszatérés előtt töröld a felülírásokat a `Remove-Item`

`Env:LLM_PROVIDER, Env:EMBEDDING_PROVIDER, Env:CHROMA_DIR` parancssal, vagy állítsd be kifejezetten a kívánt valódi értékeket.

CLI-parancsok

Parancs	Használat
<code>config</code>	Az érvényes beállítások kiírása környezeti változók formájában.
<code>ingest</code>	Az index igazítása a helyi korpuszhoz. A <code>--download</code> előbb letölt; a <code>--rebuild</code> újraépíti a gyűjteményt.
<code>serve</code>	A tudásbázis előkészítése és a Streamlit indítása; támogatja a <code>--address</code> és <code>--port</code> opciót.
<code>eval</code>	A teljes gráf vagy támogatott külön csomópont értékelése.
<code>loadtest</code>	Gráfkérelmek mérése korlátozott párhuzamossággal.
<code>export-graph</code>	A lefordított fő gráf, a RAG-gráf vagy mindkettő Mermaid-exportja.

```
uv run agentic-rag export-graph --output tmp/graphs.md
uv run agentic-rag export-graph --graph rag --format mermaid
```

A `--sources PATH` csak az `ingest --download` mellett használható. Nyers Mermaid-kimenethez egyetlen gráfot kell választani; a `--graph all --format mermaid` érvénytelen. Minden parancsnak saját `--help` súgója van.

05 Futtatás Docker Compose-zal

Az alapértelmezett rendszer az `app`, az `ollama` és az egyszer lefutó `ollama-pull` szolgáltatásból áll. Az alkalmazás megvárja az egészséges szervert és a modellletöltés befejezését. A felület a helyi 8501-es porton érhető el; az Ollama csak a Compose-hálózaton hozzáférhető, közzétett gazdagépi port nélkül.

```
docker compose up --build
docker compose logs -f app
```

Az `INGEST_ON_START=true` miatt az alkalmazás még a Streamlit indítása előtt letölti a forrásokat és elkészíti az indexet. Első indításkor a hosszú `health: starting` állapot valós letöltést és beágyazást jelezhet. A Dockerfile 20 perces türelmi időt ad az állapotellenőrzéshez; ez nem rögzített indulási idő.

Névesített kötet	Tárolt adatok
<code>ollama-data</code>	Letöltött beszélgetőmodellek súlyai.
<code>corpus-data</code>	Korpusz a <code>/app/data/raw</code> útvonalon.
<code>chroma-data</code>	Indexek a <code>/app/data/chroma_db</code> alatt, beágyazási szolgáltatónként elkülönítve.
<code>hf-cache</code>	Hugging Face-modellgyorsítótár.

Alternatív indítások

```
$env:LLM_PROVIDER = "fake"
$env:EMBEDDING_PROVIDER = "fake"
docker compose up --build --no-deps app
```

NVIDIA-támogatáshoz: `docker compose -f compose.yaml -f compose.gpu.yaml up --build`. Windowson Docker Desktop WSL 2 háttérrendszer és GPU-támogatás kell. A kiegészítő fájl nélkül az Ollama-konténer CPU-n fut.

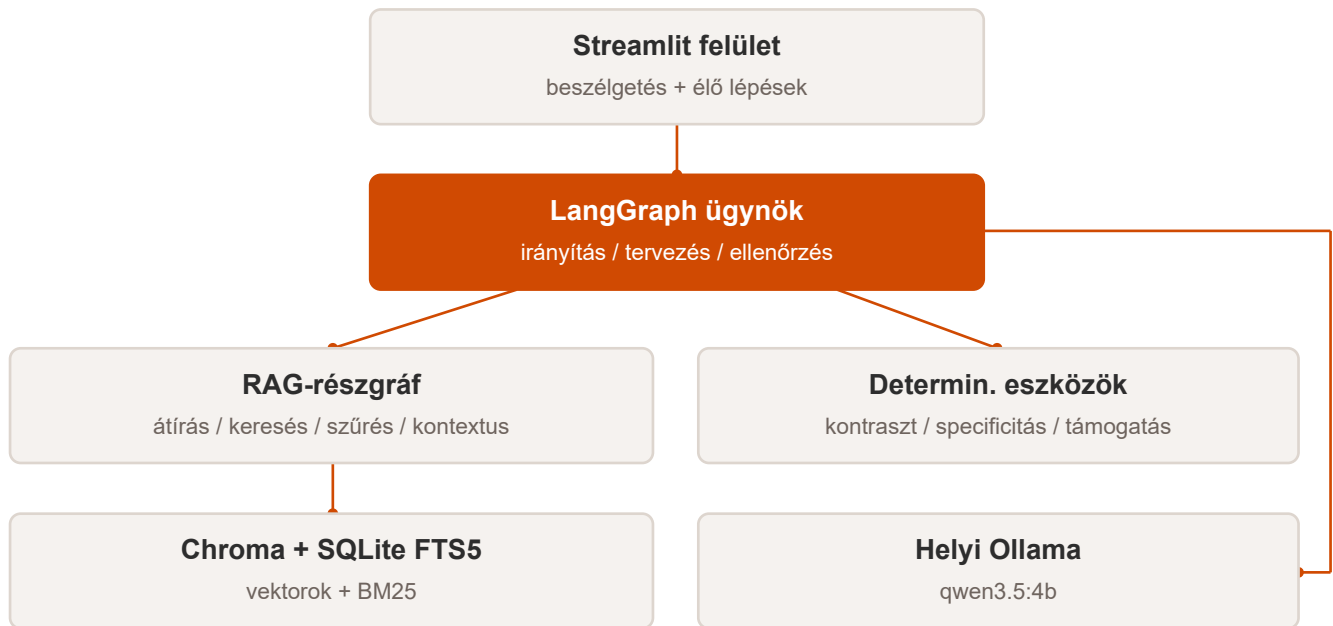
Már működő gazdagépi Ollama esetén:

```
docker compose run --rm --no-deps --service-ports `
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 app
```

Az alkalmazás alapértelmezett UID/GID értéke 10001. Linuxon a közvetlenül csatolt könyvtáraknak írhatónak kell lenniük ezzel az azonosítóval; maga a rendszer névesített köteteket használ. Az értékelést a gazdagépen futtasd, ha nem csatolod külön az adatkészletet és az eredménykönyvtárat.

A `docker compose down` megtartja a köteteket. A `-v` kapcsoló törli a korpuszt, az indexeket és a letöltött modelleket. Forrás: `compose.yaml`, `compose.gpu.yaml`, `Dockerfile`.

06 Futásidejű architektúra



A Streamlit felület a beszélgetés üzeneteit adja át a lefordított fő gráfnak. Az ügynök osztályozza a kérést, szükség esetén tervet készít, keresést vagy eszközöket hív, választ állít össze, majd véglegesítés előtt ellenőrzi azt.

Függőségi határok

A `build_agent_graph(settings)` egyszer köti be a függőségeket kulcsszavas argumentumokkal és `functools.partial` használatával. A csomópontok egyetlen pozicionális argumentuma az állapot; részleges állapotfrissítést adnak vissza. Az irányítófüggvények modellt nem hívnak, és állapotot nem módosítanak.

A `build_rag_graph(settings)` késleltetetten megnyitott vektortárat biztosít. Az első inicializálást zárolás védi, így a párhuzamos keresések ugyanazt a beágyazási modellt és indexet használják. A BM25-pillanatkép is késleltetetten, zárolás alatt épül fel.

A gráfok létrehozása nem tölt le modellt, nem kapcsolódik az Ollamához, és nem nyitja meg az indexet. Ez olcsóvá teszi a gráfexportot és a szerkezeti teszteket. Az első tényleges keresés viseli a betöltés költségét.

A csomópontok szinkron függvények. A LangGraph szálakon hajtja végre a párhuzamos `Send` feladatokat; a felület, az értékelés és a terheléses mérés ugyanezt a szinkron végrehajtási utat használja. Egy csomópont `async def`-re alakításakor a hívókat is át kell vizsgálni.

Forrás: `agent/graph.py`, `rag/graph.py`, `rag/lexical.py`. Az ábra hívási és függőségi kapcsolatokat mutat; a Python-komponensek között nincs külön hálózati API.

07 Irányítás, tervezés és ellenőrzés

Szándék	Végrehajtási út
direct	Az irányító beszélgetési választ készít, amelyet azonnal a véglegesítés követ.
single	Az irányító egy keresési részfeladatot készít, amely közvetlenül a RAG-végrehajtóhoz kerül.
tool	Az irányító egy eszközfeladatot készít, amely közvetlenül az eszköz-végrehajtóhoz kerül.
complex	A tervező legfeljebb öt önálló részfeladatra bontja a kérést.

A `plan_subtasks` típusos feladatokat állít elő. A `dispatch_subtasks` feladatonként egy `Send` objektumot küld a `run_rag_subtask` vagy `call_tool` csomópontnak. Az eredmények összegyűjtése után körönként egyszer fut a `synthesize_answer`. Hiányzó szándék vagy egylépéses terv esetén a végrehajtás tervezésre vált.

Az ellenőrzés eredményei

- `grounded`: az alátámasztott választervezet véglegesíthető.
- `insufficient`: újratevezés, amíg `retry_count < MAX_RETRIES`; a határ elérése után a válasz kifejezetten jelzi a részlegességet és a hiányzó bizonyítékot.
- `unavailable`: a strukturált ellenőrzési válasz nem olvasható. A rendszer visszatartja a tervezetet és újrapróbálást kér; nem állítja, hogy az ellenőrzés sikerült.

A `MAX_RETRIES=2` az első körön túl két újratevezést enged. Ez különbözik a négy LLM-csomópont és a RAG-végrehajtó átmeneti hibáira érvényes `RetryPolicy(max_attempts=3)` szabálytól. A programozási hibák nem számítanak szokásos átmeneti hibának.

Egyetlen pontos eszközeredmény gyors útja

Egy sikeres beépített eszköz eredményét a válaszösszeállítás modellhívás nélkül adja tovább; az irányítás az LLM-ellenőrzést is kihagyja. Az engedélyezett eszközök: `check_contrast`, `css_specificity`, `browser_support`. A szándék felismeréséhez továbbra is kell egy modellhívás.

A gyors út feltétele az egyező feladat és eredmény, a nem üres kimenet, a siker és a keresési források hiánya. Sikertelen, ismeretlen, többszörös vagy kereséssel vegyes eszközhívásnál összeállítás és ellenőrzés következik. A véglegesítés szó szerint megőrzi a sikeres eszközkimeneteket, az ismétlődő azonos kimeneteket egyszer mutatja.

Megvalósítás: `agent/graph.py`, `agent/routing.py`, `agent/nodes.py`, `agent/prompts.py`.

o8 Állapotszerződések és Python API

A fő gráf nyilvános bemenete az `AgentInput`, amely `messages` mezőt tartalmaz. A kimenet része az `answer`, `sources`, `messages`, `question`, `intent`, `subtask_results` és `trace`. A terv és az ellenőrzés mezői útvonalfüggők; közvetlen válasznál nem feltétlenül van ellenőrzési ítélet.

```
from agentic_rag.agent.graph import build_agent_graph
from agentic_rag.config import Settings

settings = Settings(
    _env_file=None,
    llm_provider="fake",
    embedding_provider="fake",
)
graph = build_agent_graph(settings)
result = graph.invoke({"messages": [{"user", "Hello"}]})
print(result["answer"])
```

Ekhez a köszönési példához nem kell index. Dokumentációs kérdéshez a kiválasztott beágyazásokkal egyező index szükséges. A könyvtári függvények kifejezetten kapják meg a beállításokat; a `get_settings()` a belépési pontok és a tesztfixture-ök számára van fenntartva.

Reducerek és megváltoztathatatlan feladatok

A `messages` az `add_messages` reduercélú függvényt használja; a `subtask_results` és a `trace` listái hozzáfűzéssel bővülnek. A többi mezőt az új érték felülírja. A csomópont részleges frissítést adjon vissza, ne módosítsa a bemeneti állapotot.

A `Subtask` mezői: `id`, `kind`, `input`, `tool_name`, `tool_args`. Eszközfeladatnál eszköznév kötelező. A `SubtaskResult` tárolja a feladatazonosítót, a kimenetet, a rangsorolt forrásokat, a siker/hiba állapotot és az eszköznevet. Az `error` pontosan akkor legyen kitöltve, ha az `ok` hamis. Ezek megváltoztathatatlan Pydantic-modellek.

Minden tervezési kör az `Overwrite(kept)` használatával visszaállítja az eredménylistát. Egyszerű hozzáfűzésnél az elutasított kör bizonyítékai a következő válaszba keverednének. A tervező megtarthat megbízható eredményeket, és elhagyhatja az újbóli futtatásukat.

Hivatkozások és beszélgetési előzmények

Az `[n]` jelölő a `sources[n - 1]` elemre mutat. A RAG-feladatok helyben számoznak; a válaszösszeállítás közös számozást készít és chunkazonosító szerint kiszűri az ismétlődéseket. A véglegesítés törli a tartományon kívüli jelölőket.

Nincs ellenőrzőpont-kezelő. A hívó minden alkalommal átadja a szükséges előzményeket. Tartós állapot hozzáadásakor a körönkénti mezőket vissza kell állítani, és a hívóknak csak az új üzeneteket szabad küldeniük, különben duplikálódhatnak az előzmények.

09 Korpuszletöltés és előfeldolgozás

A hiteles forráslista a `data/sources.toml`. Minden `[[sources]]` bejegyzés azonosítót, megjelenített nevet, adattárat, teljes commit SHA-t, licencet, gyökérkönyvtárat, bevonási/kizárási mintákat és nyilvános URL-sablont ad meg. A rögzített commit megismételhető korpuszt biztosít a legfrissebb dokumentáció észrevétlen követése helyett.

```
uv run agentic-rag ingest --download
uv run agentic-rag ingest --download --sources data/sources.toml
```

A letöltő részleges Git-kikéréssel másolja a kiválasztott fájlokat a `DATA_DIR/<id>/` könyvtárba. A `.source.json` jegyzék tárolja a forrás metaadatait. Egyező meglévő jegyzék esetén a letöltés kihagyható; a commit vagy a kiválasztás módosítása a következő letöltéskor érvényesül.

A forrásdialektusok átalakítása

A betöltők Markdown-, MDX- és egyszerű szövegfájlokat támogatnak. Megőrzik a front matter címeit, és normalizálják az MDN-makrókat, a JSX-elemeket, a VitePress-konténereket és a Nuxt MDC-komponenseit. A hivatkozások olvasható szöveggé alakulnak; a bekerített kódblokkok a tisztítás során változatlanok maradnak. A Next.js Pages Router blokkjai kimaradnak, ahol az aktuális App Router korpusz ezt igényli.

Az oldalak H2/H3 szakaszokra bomlanak; a címsorútvonal a `section` metaadatba kerül. A ponttal kezdődő fájlok és könyvtárak, valamint a README-fájlok kimaradnak. Nem támogatott fájlformátum esetén a betöltés hibával leáll.

Szerkezetet figyelembe vevő darabolás

- Alapértelmezett törzsméret: 900 karakter; átfedés: legfeljebb 150 karakter.
- A bekezdések és szerkezeti blokkok együtt maradnak; a címsor az általa bevezetett szöveggel kerül egy darabba.
- A kódblokkok 1800 karakterig egyben maradnak; a nagyobb blokkok és bekezdések további bontást kapnak.
- Minden darab elején megjelenik a cím és a szakaszútvonal, így az elszigetelt szöveg is azonosítja az API-t és a keretrendszert.

A darabok megőrzik a `source`, `title`, `section`, `url` és az elérhető `revision` mezőt, valamint saját azonosítót és eltolásokat kapnak. A `browser-compatible data index=false` beállítással töltődik le: eszközbemenet, nem beágyazandó szöveg.

Megvalósítás: `ingestion/sources.py`, `download.py`, `markdown.py`, `loaders.py`, `chunking.py`.

10 Az index életciklusa és migrációja

A Chroma helyi, tartós vektortár. Az alapértelmezett gyűjtemény a `documents`; a koszinusztávolság `1 - distance` relevanciapontszámmá alakul. A beágyazási szolgáltató és modell metaadatai védik a gyűjteményt az inkompatibilis keresési vektoroktól.

Szokásos szinkronizálás

```
uv run agentic-rag ingest
```

A betöltés beágyazza az új vagy módosult darabokat, majd az aktuális korpusz indexelése után törli az elavult azonosítókat. Változatlan dokumentumoknál az ismételt futás inkrementális. Üres korpusz esetén hiba keletkezik; a rendszer nem törli ki a meglévő indexet.

Beágyazási modell cseréje

```
uv run agentic-rag ingest --rebuild
```

Ugyanazon gyűjtemény használatakor a `EMBEDDING_PROVIDER` vagy `EMBEDDING_MODEL` módosítása után `--rebuild` szükséges. Eltérő modellek vektorai nem keverhetők. Közvetlen indexmegnyitáskor eltérés esetén `EmbeddingMismatchError` keletkezik; az induláskori előkészítés felismeri ezt az állapotot és automatikusan újraépít.

Az újraépítés lecseréli a kiválasztott gyűjteményt. Kísérletekhez külön `CHROMA_DIR` értéket használj, ha meg akarsz őrizni a működő alapváltozatot. Csak a beszélgetőmodell cseréje nem igényel új vektorindexet.

A megbízhatósági frissítés migrációja

A jelenlegi betöltés a letöltött források commit SHA-ját a darabok metaadataiba és azonosságába is beépíti. Régebbi indexhez egyszer futtasd a `sima ingest` parancsot, majd indítsd újra az alkalmazást. Az első migráció újra beágyazza az érintett darabokat, mert változik az azonosítójuk. A későbbi változatlan futások inkrementálisak maradnak.

A nyilvános dokumentációs URL újabb tartalmat is mutathat, mint az indexelt pillanatkép. A `Source.revision` az indexelt commitot azonosítja; nem garantálja, hogy a nyilvános URL ezt a verziót szolgálja ki. A verziómetaadat nélküli korábbi helyi dokumentumok továbbra is használhatók.

A lefordított gráf gyorsítótárazza a vektortár-szolgáltatót és a memóriában lévő BM25-pillanatképet. A korpusz módosítása vagy újraindexelése után indítsd újra az alkalmazást, vagy építs új gráfot. A meglévő lexikai pillanatkép nem frissül automatikusan.

Megvalósítás: `ingestion/index.py`, `ingestion/prepare.py`, `rag/lexical.py`; migráció: `docs/rag-improvements.md`.

11 Hibrid RAG-keresés

A RAG-gráf sorrendje mindig `rewrite_query`, `retrieve`, `grade_documents`, `build_context`. Nyilvános bemenete `{"query": ...}`; kimenete hivatkozott kontextust, rangsorolt forrásokat és nyomkövetési eseményeket tartalmaz. Az újratervezést a fő ügynök végzi, nem belső RAG-ciklus.

Átírás és keresési jelöltek

Ollamával az átírás egy rövid angol keresőkérdést készít, megőrizve az API- és keretrendszerneveket. Üres vagy túl hosszú válasznál az eredeti kérdés marad. Fake módban ez a modellhívás kimarad.

A jelöltilista mélysége `max(TOP_K, RETRIEVAL_CANDIDATES)`, jelenleg 20. A Chroma szemantikus találatokat ad. `HYBRID_SEARCH=true` esetén SQLite FTS5 BM25-találatok egészítik ki ezeket ugyanazon indexelt darabokból. A címek és szakasznevek nagyobb súlyt kapnak a törzsszövegnél.

A lexikai lekérdezés az eredeti és az átírt szöveget is tartalmazza. A tokenek idézőjelbe kerülnek, az SQL-paraméterek kötöttek. Így a `:has`, `$fetch`, az aláhúzás és a kötőjel megőrizhető anélkül, hogy a felhasználó szövege FTS-parancsként értelmeződne.

Rangsorfúzió, szűrés és kontextus

A reciprok rangfúzió a szemantikus és lexikai listákban összeadja az $1 / (60 + \text{rank})$ értékeket, majd kiválasztja a jelölteket. A fúziós rang különbözik a koszinusz hasonlóságtól. A kizárólag kulcsszavas találat első pontszáma -1; a kimeneti forrás hasonlósága kitalált érték helyett `None`.

A kezdeti koszinuszküszöb Hugging Face esetén 0,83, fake beágyazásnál 0,0. A lexikai találatok megkerülhetik ezt a küszöböt, de a bekapcsolt LLM-szűrő ezeket is vizsgálja. Egy strukturált hívás értékeli az összes megmaradt jelöltet; a végső lista legfeljebb `TOP_K=4` darabot tartalmaz.

Olvashatatlan relevanciaértékelésnél megmaradnak a küszöbön vagy kulcsszavakon átjutó jelöltek, és figyelmeztetés kerül a naplóba. Kapcsolati vagy időtúllépési hiba a szülő részfeladat átmeneti újrapróbálási szabályához jut. Ez eltér a végső válaszellenőrzéstől: annak olvashatatlan eredménye visszatartja a tervezetet.

A `build_context` kiszűri az azonos szövegeket és helyi jelölőket rendel hozzájuk. Bizonyíték nélkül üres kontextust és forráslistát ad. A hasonlóság és a rang keresési jelzés, nem a válasz megbízhatóságának becslése.

Megvalósítás: `rag/graph.py`, `rag/nodes.py`, `rag/lexical.py`.

12 Determinisztikus eszközök

Az eszközök Pydantic-argumentumsémával rendelkező LangChain `BaseTool` példányok. A tervező kifejezett eszköznevet és argumentumokat választ a modell natív eszközhívásaira hagyatkozás helyett. A beépített determinisztikus bemeneti sémák elutasítják az ismeretlen mezőket.

Eszköz	Bemenet és működés
<code>search_knowledge_base</code>	<code>query</code> : meghívja a RAG-részgráfot, hivatkozott szöveget és RAG-eredményt ad vissza.
<code>check_contrast</code>	<code>foreground</code> , <code>background</code> : támogatott CSS-színekből WCAG-kontrasztarányt és AA/AAA eredményt számol. A háttér legyen átlátszatlan.
<code>css_specificity</code>	<code>selectors</code> : 1-10 szelektor. A Selectors Level 4 alapján kiszámítja és összehasonlítja az (a, b, c) értékeket.
<code>browser_support</code>	<code>feature</code> , opcionális <code>area</code> és <code>browsers</code> : megkeresi a webes funkciót a rögzített MDN-adatokban és összeveti a megadott böngészőverziókkal.

Közvetlen meghívási példa

```
from agentic_rag.agent.tools import get_non_retrieval_tools
from agentic_rag.config import Settings

tools = {tool.name: tool for tool in
          get_non_retrieval_tools(Settings(_env_file=None))}
print(tools["check_contrast"].invoke({
    "foreground": "#777777", "background": "#ffffff"
}))
```

A kontraszt- és specificitás számításához nem kell LLM. A böngészőtámogatáshoz a `DATA_DIR` alatt letöltött `browser-compat-data` kell. Ez késleltetetten töltődik be; hiányzó korpusz miatt az első támogatási hívás akkor is hibázhat, ha a többi eszköz működik.

Az eredmények helyes értelmezése

A kontraszteredmény a megjelenítési kerekítés előtt készül; az arány lefelé kerekítve jelenik meg. A specificitás azonos eredetű, fontosságú és kaszkádrétegű deklarációk között dönt; a teljes kaszkádban az `!important`, az inline stílusok és a rétegek is számítanak. A böngészőtámogatás a rögzített adatállományt és annak feltételeit tükrözi, nem élő böngészőtesztet.

Validációs vagy szakterületi hiba sikertelen feladateredménnyé alakul. A véglegesítés szó szerint megőrzi a sikeres kimenetet és annak nyelvét. Kereséssel vegyes eszközkérséknél továbbra is szükséges a válaszösszeállítás és ellenőrzés.

Megvalósítás: `agent/tools.py`, `contrast.py`, `specificity.py`, `compat.py`.

13 Modellek és tárolás beállításai

Minden futásidejű beállítás a megváltoztathatatlan `Settings` modellen keresztül érvényesül. A környezeti változók felülírják a `.env` értékeit; az üres érték alapértelmezést jelent, az ismeretlen változó kimarad. A relatív útvonalak az aktuális munkakönyvtárból indulnak. A `.env` legyen UTF-8, különösen Windows PowerShell 5.1 használatakor.

Változó	Alapérték	Jelentés / korlát
LLM_PROVIDER	ollama	ollama vagy az előre megírt fake.
OLLAMA_BASE_URL	http://localhost:11434	Gazdagépi szervercím; Compose-ban <code>http://ollama:11434</code> .
OLLAMA_MODEL	qwen3.5:4b	Helyi beszélgetőmodell-címke; gazdagépi futás előtt töltsd le.
OLLAMA_NUM_CTX	8192	Közös prompt- és válaszlak; 512-131072 token.
OLLAMA_TIMEOUT_S	120.0	Pozitív, véges HTTP-időtűllépés másodpercben, Ollama-kérésenként.
OLLAMA_REASONING	false	Gondolkodási mód; az ezt nem támogató modellek figyelmen kívül hagyják.
LLM_TEMPERATURE	0.0	Mintavételi hőmérséklet 0,0 és 2,0 között.
EMBEDDING_PROVIDER	huggingface	Helyi sentence-transformers vagy fake.
EMBEDDING_MODEL	intfloat/multilingual-e5-small	Indexeléshez és kereséshez ugyanaz a modell szükséges.
DATA_DIR	data/raw	Letöltött és helyi forráskorpusz.
CHROMA_DIR	data/chroma_db	Tartós vektorindex könyvtára.
CHROMA_COLLECTION	documents	Projektkorlát: 3-63 betű, szám, pont, aláhúzás vagy kötőjel; betűvel/számmal kezdődik és végződik, nincs <code>..</code> , nem IPv4-cím.

A többnyelvű E5-modell CPU-n fut; a keresési és dokumentumprefixeket az `embeddings.py` kezeli. Az alapmodell 384 dimenziós vektorokat készít. Modellcseréhez kompatibilis új index szükséges.

A Compose a korpusz- és indexútvonalakat a csatolt konténerútvonalakkal felülírja. A gazdagépi `.env` útvonala nem helyezi át ezeket a köteteket. Nagyobb kontextus több modell- és KV-gyorsítótár-memóriát igényel; önmagában nem javítja a relevanciát.

Hiteles definíciók: `config.py`, `.env.example`, `embeddings.py`, `compose.yaml`. Az érvényes értékek: `uv run agentic-rag config`.

14 A munkafolyamat beállításai

Változó	Alapérték	Jelentés / korlát
TOP_K	4	A szűrés után megtartott releváns darabok maximuma keresésenként; legalább 1.
RETRIEVAL_CANDIDATES	20	Szűrés előtti jelöltlista; 1-100. A tényleges mélység legalább TOP_K.
HYBRID_SEARCH	true	Vektorkeresés és helyi SQLite FTS5 BM25 összefésülése.
GRADE_WITH_LLM	true	Egy relevanciaértékelő hívás keresésenként; fake LLM-nél kimarad.
MAX_RETRIES	2	Ellenőrzés miatti újratervezések maximuma; legalább 0. Nulla esetén nincs újratervezés.
INGEST_ON_START	true	A <code>serve</code> a felület előtt letölti a hiányzó forrásokat és szinkronizálja az indexet.
LOG_LEVEL	INFO	DEBUG, INFO, WARNING vagy ERROR.

Ellenőrzött keresési összehasonlítás

A korábbi, csak vektoros keresési konfiguráció közelítéséhez PowerShellben:

```
$env:HYBRID_SEARCH = "false"
$env:RETRIEVAL_CANDIDATES = "4"
$env:TOP_K = "4"
uv run agentic-rag eval --dataset data/eval/questions.jsonl
```

A jelenlegi hibrid alapértékekhez `HYBRID_SEARCH=true` és `RETRIEVAL_CANDIDATES=20` kell. Ugyanazzal a korpussszal, beszélgetőmodellel és bírálóval ismételd a mérést. Ezek a beállítások a keresési konfigurációt közelítik, nem a teljes történeti kódműködést: az ellenőrzés és az eszközök gyors útja is változott.

Beállítások könyvtári használata

A beállítások rögzítettek és gyorsítótárkulcsként használhatók. Ellenőrzött változatot a `Settings.model_validate(**settings.model_dump(), **changes)` készít. A `model_copy(update=...)` kerülendő, mert kihagyja a validációt.

A `get_settings()` folyamatonként egy példányt gyorsítótáraz. Ha egy teszt szándékosan módosítja a környezetet, törölje ezt a gyorsítótárat. Futó felületnél a környezet- és korpuszváltozások biztos alkalmazásához újraindítás ajánlott.

A `MIN_SCORES` küszöbei és a darabolási alapértékek kódbeli konstansok vagy konfigurációs objektumok, nem további környezeti változók. Új beállításnál együtt frissítsd a `Settings` modellt, a `.env.example` fájlt, a teszteket és az érintett konténerbeállításokat.

Hiteles definíciók: `config.py`, `rag/graph.py`, `ingestion/chunking.py`.

15 Felület és nyomkövetés

A Streamlit alkalmazás munkamenet-állapotban tárolja a beszélgetést és a beállítások szerint gyorsítótárazza a lefordított gráfot. A lezárt előzményeket és az új felhasználói üzenetet adja át. A megszakított kör megjelenik, de a következő modellbeszélgetésbe nem kerül be.

Élő végrehajtási lépések

A felület szinkron gráfráfrissítéseket fogyaszt, bekapcsolt részgráf-streameléssel. A fő lépések befejezéskor jelennek meg; a párhuzamos feladatok LangGraph-lépés szerint csoportosulnak. Minden keresésnél látható lehet az átírás, a jelöltek és a megtartott bizonyíték. A kész nyomkövetés alapról összezsukott, a futó nyitva látható.

Ez a munkafolyamat lépéseinek streamelése. A jelenlegi felület a gráf befejezése után mutatja a végső választ; nem streameli egy még ellenőrizetlen tervezet tokenjeit. A forráskártyák megmutatják az indexelt kontextust és az elérhető commitverziót.

A számozott hivatkozások kattintható HTTP(S) referencialinkekké alakulnak. A hivatkozásformázás megőrzi a kód példákat. A nyilvános URL navigációs link, az indexelt verzió pedig a bizonyíték pillanatképét azonosítja.

Közös mérési infrastruktúra

A `@traced` minden sikeresen befejezett csomóponthoz `TraceEvent` eseményt készít. Mezői: `node`, `started_at`, `ended_at`, `duration_ms`, `summary`, `metadata` és opcionális `step`. Az időmérés monoton, nagy felbontású órát kapcsol az epoch-időhöz.

A `run_rag_subtask` továbbítja a részgráf eseményeit a fő nyomkövetésbe. Élő részgráf-streameléskor a `trace_events_from_chunk(..., skip_forwarded=True)` megakadályozza, hogy a szülő frissítése ismét megjelenítse ugyanazokat az eseményeket.

A szülő RAG-feladat idejét és gyermekcsomópontjai idejét ne add össze független kiszolgálási időként. A párhuzamos időtartamok átfedhetnek; összegük eltér a teljes kérés tényleges idejétől.

Korlátok bővítéskor

A dekorátor szótáras részleges frissítést vagy `None` értéket támogat. LangGraph `Command` visszaadásához külön nyomkövetési támogatás kell. Csomópontszintű `CachePolicy` régi időbélyegeket játszana vissza; a belső munkát gyorsítótárazd, hogy minden végrehajtás friss eseményt adjon.

Megvalósítás: `ui/app.py`, `ui/components.py`, `tracing.py`; tartós munkamenettár nincs beállítva.

16 Tesztelés és folyamatos integráció

A tesztek fake szolgáltatókat, ideiglenes adatokat és ellenőrzött beállításokat használnak. Az alapértelmezett pytest-konfiguráció kizárja az élő `ollama` jelölőt. Így a tesztkészlet GPU, élő modell és letöltött frontend-korpusz nélkül futtatható offline.

```
uv run ruff check .
uv run ruff format --check .
uv run pytest
```

Fejlesztés közben az érintett tesztfájlt futtasd, összeolvasztás előtt pedig a szükséges teljes ellenőrzéseket. A letöltési tesztek ideiglenes helyi Git-adattárakat hoznak létre; Git hiányában kimaradhatnak.

Fontos tesztterületek

Tesztfájlok	Ellenőrzött viselkedés
<code>test_config.py</code> , <code>test_cli.py</code>	Alapértékek, validáció, elsőbbség, opciók és kilépési kódok.
<code>test_ingestion.py</code> , <code>test_embeddings.py</code>	Tisztítás, darabhatárok, inkrementális index és modellazonosság.
<code>test_rag_subgraph.py</code> , <code>test_rag_improvements.py</code>	Keresés/szűrés, hibrid fúzió, tartalékágak és metaadat-migráció.
<code>test_agent_graph.py</code> , <code>test_state.py</code>	Irányítás, párhuzamos szétosztás, reducerek, korlátos újratervezés és ellenőrzés.
<code>test_tools.py</code> , <code>test_ui.py</code> , <code>test_tracing.py</code>	Pontos eszközeredmények, felületi megjelenítés és eseménytovábbítás.
<code>test_evaluation.py</code> , <code>test_loadtest.py</code>	Szigorú adatkészletek, metrikák, összesítés és jelentések.

Élő modellellenőrzés

```
ollama pull qwen3.5:4b
uv run pytest -m ollama
```

Az élő teszt a gazdagépi Ollama-beállításokat használja; elérhetetlen szervernél kimaradhat. Kapcsolati és modellintegrációs ellenőrzés, a válaszok funkcionális minőségét külön az `eval` méri.

A CI minden pull requestnél és a `main` ágra történő push esetén fut. Telepíti az `uv` 0.12.6 verzióját, szinkronizálja a lockfájlt, ellenőrzi a lintet és formázást, majd futtatja az offline teszteket. Külön feladat építi a Docker-képet és meghívja benne az `agentic-rag --version` parancsot. Korábbi tesztarabszám nem tekintendő aktuális garanciának.

Forrás: `pyproject.toml`, `tests/conftest.py`, `.github/workflows/ci.yml`.

17 Az alkalmazás bővítése

Új működés hozzáadásakor őrizd meg a kifejezett függőségeket, a típusos bemeneteket és a bizonyítékezelés szerződéseit. Először a legkisebb érintett modult módosítsd, majd ellenőrizd az oda vezető útvonalat.

Dokumentációs forrás hozzáadása vagy frissítése

1. Adj a `data/sources.toml` fájlhoz rögzített `[[sources]]` bejegyzést licenccel, fájlmintákkal és nyilvános URL-leképezéssel.
2. Ellenőrizd, hogy a Markdown-tisztítók támogatják-e a dialektust; szükség esetén adj célzott betöltőtesztet.
3. Futtasd az `ingest --download` parancsot, vizsgáld meg a metaadatokat és próbálj jellemző kereséseket.
4. Indítsd újra a gráfot/felületet a lexikai pillanatkép frissítéséhez; adj jellemző fejlesztési értékelőkérdéseket.

Determinisztikus eszköz hozzáadása

A szakterületi műveletet modelltől függetlenül valósítsd meg. Készíts hozzá `BaseTool` burkolót szigorú Pydantic-bemenettel és felhasználóbarát `ToolException` hibákkal. A `get_non_retrieval_tools(settings)` útján regisztráld, hogy az irányítás és a tervezés is elérje.

Vizsgáld át a promptokat és a fake szolgáltató új útvonalát; teszteld a helyes hívást, az érvénytelen argumentumokat és a végrehajtási hibákat. A gyors út kifejezett háromelemű engedélylistát használ: egy új eszköz nem kerül meg automatikusan az ellenőrzést. Csak megfelelő, tesztelt esetben bővítsd ezt a listát.

Gráfcsomópont vagy állapotmező hozzáadása

Frissítsd az állapotszerződést, készíts szinkron csomópontot kulcsszavas függőségekkel, alkalmazz nyomkövetést, kösd be a gráfot, majd exportáld a Mermaid-ábrát. Új `Send` végrehajtónál külön add meg a bemeneti sémát. Párhuzamos írásokhoz tudatos reducer vagy külön mezők szükségesek.

`Command`, ellenőrzőpont, módosított hivatkozásszámozás vagy csomópont-gyorsítótár bevezetése előtt vizsgáld felül a 08. és 15. fejezet állapot-visszaállítási és nyomkövetési feltételezéseit.

Modellek vagy függőségek cseréje

Beszélgetőmodellek összevetéséhez azonos bírálót és adatkészletet használj. Beágyazásimodell-cseréhez új index és a küszöbök újraértékelése kell. Függőségfrissítéskor a `pyproject.toml` és az `uv.lock` maradjon összhangban; a helyi ellenőrzések mellett a konténerépítés is sikerüljön.

Bővítési pontok: `agent/tools.py`, `agent/prompts.py`, `llm.py`, `agent/graph.py`, `config.py`, `data/sources.toml`.

18 Funkcionális értékelés

A szigorú UTF-8 JSONL-betöltő soronként egy kérdést ellenőriz. Kötelező az `id`, `question`, `reference_answer`. Opcionális a várt szándék/dokumentumok, bizonyítékcsoportok, rögzített előzmény, címkék és megjegyzés. Ismételt azonosító vagy kulcs, ismeretlen mező és hibás érték fájl- és sorszámot jelző hibát okoz.

A fejlesztési regressziós készlet 17 kérdéses. A külön, 24 kérdéses holdout követő kérdéseket, magyar kérdéseket, többértelmű API-kat, verzióhatárokat, nem támogatott API-kat és vegyes eszköz/keresési kéréseket tartalmaz. Ne használd promptfinomításra; a referencia-választervezeteket szakértőnek kell ellenőriznie, mielőtt minőségi mércének tekinted.

```
uv run agentic-rag eval --dataset data/eval/questions.jsonl --judge-model qwen2.5:7b-instruct
uv run agentic-rag eval --dataset data/eval/holdout.jsonl --judge-model qwen2.5:7b-instruct
uv run agentic-rag eval --target node --node analyze_request
uv run agentic-rag eval --target node --node run_rag_subtask
```

Élő mérés előtt töltsd le a kiválasztott bírálómodellt. A bíráló opciója csak teljesgráf-értékeléshez használható. Az elkülönített keresés nem tud előzményt feloldani és elutasítja az ilyen eseteket; követő kérdéshez a teljes gráfot használd.

Metrika	Értelmezés
Routing accuracy	A kiválasztott szándék egyezik az elvárttal.
Retrieval hit@k	Legalább egy keresési feladat első k megtartott darabjában szerepel várt dokumentum.
Complete evidence@k	Minden szükséges bizonyítékcsoport megjelenik; csoporton belül a dokumentumok alternatívák.
Answer correctness	Bírálói összevetés a referenciával: 1, 0,5 vagy 0.
Faithfulness	Állítások alátámasztása keresési/eszközbizonyíték alapján: 1, 0,5 vagy 0.

A keresési metrikák a szűrés utáni, rangsorolt feladatforrásokat használják, nem a végső hivatkozáslistát. Az átlagok mellett vizsgáld az alkalmazható esetek számát és az egyedi hibákat. Fake módban nincs bírálói helyesség/hűség értékelés, így valós válaszmínőség sem állapítható meg.

A jelentések időbélyeges JSON- és Markdown-fájlok a `data/eval/results/` vagy a `--output-dir` könyvtárban. Beállításokat és eseteredményeket rögzítenek. Érdemi összevetéshez azonos bíráló, korpusz és modellkonfiguráció kell.

Mért eredmények

A 19. fejezet összeveti a jelenlegi alapbeállítások 2026. október 4-i mérését a 3-i alapértékkel, és összefoglalja a holdoutot. Minden szám a `data/eval/results/` egy jelentéséből származik.

Jelentések: fejlesztési készlet `eval-graph-20261004T191858Z`, ...`192448Z` és ...`193039Z` (azonos pontszámok); holdout `eval-graph-20261004T194127Z`; terhelés `loadtest-ollama-c4-20261004T195038Z` és `loadtest-ollama-c1-20261004T195534Z`.

19 Teljesítmény és mért eredmények

A RAG-megbízhatósági frissítést 2026. október 4-én mértük a 3-i alapértékkel szemben, ugyanazon a gépen (Windows 11, 8 GB-os RTX 5070 Laptop GPU, Ollama 0.35.0 alapbeállításokkal, CPU-s E5-beágyazás) és ugyanazokkal a modellbuildekkel: `qwen3.5:4b 2a654d98e6fb` kikapcsolt gondolkodással, bíráló a `hf.co/bartowski/Qwen2.5-7B-Instruct-GGUF:Q4_K_M`. Az Ollama-tagek változhatnak; összevetés előtt nézd meg az `ollama list` kimenetét.

Funkcionális minőség

Mérés	Alapérték	Frissítés
Irányítás / hit@4 (17 kérdés)	1,00 / 0,90	1,00 / 1,00
Helyesség / hűség	0,91 / 0,94	0,91 / 1,00
Holdout, 24 kérdés: helyesség / hűség	–	0,67 / 0,73

A helyesség azért marad 0,91, mert a bíráló két helyes választ félreolvas (nélkülük 0,97); a holdouton három bírálói hiba 0,12-t takar el (0,79). A holdout a verzióhatároknál, egy kétértelmű keretrendszer-kérdésnél és a válasz nyelvénél hibázik. `HYBRID_SEARCH=false` és `RETRIEVAL_CANDIDATES=4` mellett a keresés visszaesik 0,90-re.

Terhelés

Terheléses futás (egy slot)	Áteresztés	p50 / p95
Alapérték, 1 felhasználó, 50 kérés	11,5/perc	3,6 s / 14,9 s
Frissítés, 1 felhasználó, 50 kérés	11,7/perc	3,9 s / 12,9 s
Alapérték, 4 felhasználó, 100 kérés	12,2/perc	17,6 s / 35,4 s
Frissítés, 4 felhasználó, 100 kérés	12,0/perc	20,4 s / 39,7 s

A szűk keresztmetszet továbbra is a sorban álló helyi LLM-dekódolás. A pontos eszközválaszok kb. 4,7 LLM-hívásra csökkentik egy kérés költségét (korábban 5,3), a 20 jelölt szűrése viszont keresésenként 0,22 s-mal hosszabb, ami terhelés alatt látszik. Több Ollama-slot a tesztelt 7B transformernek segített, ennek a Qwen3.5-buildnek nem.

```
uv run agentic-rag loadtest --requests 100 --concurrency 4
uv run agentic-rag loadtest --requests 50 --concurrency 1
```

Bizonyíték: `docs/evaluation.md`, `docs/performance.md`, `docs/rag-improvements.md`, `data/eval/results/`.

20 Hibaelhárítás és üzemeltetés

Jelenség	Teendő
Hiányzó tudásbázisindex	Futtasd az <code>ingest --download</code> parancsot; a <code>config</code> segítségével ellenőrizd a <code>DATA_DIR</code> és <code>CHROMA_DIR</code> értékeit. Előkészítési hiba után a felület index nélkül is elindulhat.
Eltérő beágyazások	Válaszd az index eredeti szolgáltatóját/modelljét, vagy futtass <code>ingest --rebuild</code> parancsot. Az induláskori előkészítés is újraépíti az eltérő indexet.
Ollama-kapcsolat elutasítva	Ellenőrizd a szerveret és a futási környezethez illő gazdagépi vagy Compose-címet az <code>OLLAMA_BASE_URL</code> változóban.
Hiányzó modell	A használt szerveren töltsd le az <code>OLLAMA_MODEL</code> modellt; a gazdagépi letöltés nem tölti fel a külön konténerkötetet.
Böngészőtámogatási hiba	Töltsd le a korpuszt és ellenőrizd a <code>DATA_DIR/browser-compat-data</code> könyvtárat. A kontraszt/specificitás ezt nem igényli.
Régi források betöltés után	Indítsd újra az alkalmazást vagy építs új gráfot a lexikai pillanatkép és erőforrások frissítéséhez.
Hibás vagy olvashatatlan <code>.env</code>	Mentsd UTF-8-ként; ellenőrizd a jelzett értékeket és az ütköző környezeti felülírásokat.
Lassú első kérés / indulás	Vizsgáld a letöltési és betöltési naplókat; különítsd el az inicializálást a már betöltött modell idejétől.
Hibás keretrendszeres válasz	Vizsgáld az átírást, a megtartott darabokat, a forrásverziót és a várt bizonyítékot.
Visszatartott választervezet	Próbáld újra, és nézd meg az ellenőrzési parse-hibák naplóját. Az <code>unavailable</code> szándékosan visszatartja a tervezetet.

Üzemeltetési parancsok

```
uv run agentic-rag config
docker compose ps
docker compose logs --tail 100 app
docker compose logs --tail 100 ollama
docker compose run --rm --no-deps app agentic-rag ingest --rebuild
```

CLI-kódok: 0 sikeres parancsvégrehajtás; 1 várt működési hiba, például hiányzó adat vagy letöltési hiba; 2 használati/konfigurációs hiba; 130 megszakítás. Sikeres értékelőparancs jelentése is tartalmazhat hibás eseteket, ezért vizsgáld a hiba mezőit.

A váratlan programozási kivételek megtartják a stack trace-t. Kerüld ezek felhasználói hibaként való elrejtését. A tartós kötetek törlése helyett először célzott adat-, modell- vagy konfigurációjavítást használj.

21 Karbantartás és forrásjegyzék

Az útmutató a projekt 0.1.0 verzióját írja le 2026. október 4-i állapotában, a RAG-megbízhatósági frissítéssel együtt. Ha egy korábbi terv vagy mérés eltér a jelenlegi működéstől, a megvalósítás az irányadó.

Hivatkozás	Mire használható?
README.md, README.hu.md	Projektcél, telepítés, tervezési döntések és mért eredmények.
docs/architecture.md	Részletes munkafolyamat-, állapot- és függőségi szerződések.
docs/rag-improvements.md	A frissítés: hibrid keresés, ellenőrzés, holdout, migráció és mért eredmények.
docs/evaluation.md	Funkcionális mérések (alapérték, frissítés, holdout), a bíráló megbízhatósága és megismétlés.
docs/performance.md	Terheléses mérések (alapérték és frissítés), csomópontidők és szűk keresztmetszet.
data/README.md, data/eval/README.md	Korpuszfelépítés, szigorú adatséma és jelentésértelmezés.
config.py, .env.example	Hiteles futásidejű beállítások és validációs szabályok.
agent/, rag/, ingestion/	Hiteles futásidejű és adatbetöltési működés.
compose.yaml, Dockerfile, .github/workflows/ci.yml	KonténERMűködés, rögzített építési eszközök és CI-ellenőrzések.

A két nyelvi változat összehangolása

Szerződés- vagy parancsváltozáskor mindkét Markdown-forrást frissítsd. Az azonosítók és parancsok maradjanak azonosak, a magyarázó szöveget fordítsd le. Ellenőrizd a `Settings` alapértékeit, a CLI `--help` kimenetét és az adatkészletek méretét, majd rendereld és vizsgáld meg mindkét PDF-et.

A források: `docs/developer-guide.en.md`, `docs/developer-guide.hu.md`. A generátor: `docs/build_developer_pdfs.py`; kimenetei: `output/pdf/developer-guide-en.pdf` és `output/pdf/developer-guide-hu.pdf`.

A dokumentumgenerátorhoz ReportLab kell, valamint Arial, Georgia és Consolas (Windows) vagy DejaVu Sans, Serif és Sans Mono betűtípus; az elrendezés a `.streamlit/config.toml` felülettémáját követi. Ezek a chatbot futásidejű függőségeitől különálló dokumentumkészítő eszközök. Megfelelő környezetben való telepítésük után a projekt gyökeréből futtasd a `python docs/build_developer_pdfs.py` parancsot.

Újragerálás után ellenőrizd az oldaltöréseket, táblázatokat, olvasható parancsokat és magyar ékezeteket. Új eredménynél nevezd meg az adatkészletet, korpuszverziót, beszélgető-/bírálómodellt és szerverbeállításokat; történeti mérést ne helyettesíts becsléssel.

A dokumentumellenőrzés az útmutatót és példáit vizsgálja; nem helyettesít új élő válaszmínőségi értékelést vagy teljesítménymérést.